EncryptEdge Labs

Junior Application Security Engineer Internship Programme

Junior Application Security Engineer

Internship Task Report

Intern Full Name	Abdoul Karim Savadogo
Intern Email	abdoulkarimsavadogo03@gmail.com
Cohort	2026
Task ID	ASE-HTTP-SESSION
Task Title	HTTP & Session Security Analysis (TLS, Cookie Manipulation, Session Threat Modelling)
Skill Domain	Web Application Security / Session Management
Difficulty Level	Beginner
CPE Credits	Per task brief
Guiding Framework(s)	OWASP Top 10:2025, CWE, 5×5 likelihood × impact risk matrix
Submission Date	15 October 2026

Copyright © 2026 EncryptEdge Labs Limited | Company No. 15830711 | United Kingdom
All activities in this report were performed within authorised lab environments as defined by the task brief.

Executive Summary

Purpose and scope. This report assesses how HTTP, TLS and cookies protect a user session, using hands-on tests from a Kali Linux workstation against httpbin.org, a public HTTP testing service. The tests were a TLS versus plain-HTTP comparison with curl (Phase A), cookie modification with Burp Suite and Postman (Phases C and D) and two TryHackMe labs on HTTP and request smuggling (Phase E).

Findings. Four findings were recorded, none High: **1 Medium, 2 Low and 1 Informational**. The service answers on plain HTTP with no redirect to HTTPS (Medium); it discloses its server version and sends a wildcard CORS header together with "allow credentials" (both Low); and it accepts and reflects any cookie value the client sends (Informational). That last result is not a session-management vulnerability in itself, because httpbin has no login and does not validate sessions. HTTPS was sound: TLS 1.3 with a certificate that verified.

Session threat model. Ten threats to session management were modelled and scored on a 5×5 likelihood × impact matrix for session hijacking (Figure 7). **Two are Critical:** sniffing or tampering with sessions over cleartext HTTP (score 20) and cookie theft through cross-site scripting (15). Four are High (cookie tampering, weak expiry and logout invalidation, session ID prediction, request smuggling) and four are Medium. The matrix assumes a typical cookie-based application. The lab showed the enabling conditions, plaintext HTTP and client-controlled cookies, but did not demonstrate a working hijack.

Priorities for a real application:

- **HTTPS everywhere:** redirect or close port 80, send HSTS and mark every session cookie Secure.
- **Server-validated sessions:** random, unpredictable IDs looked up on the server, regenerated at login, with HttpOnly and SameSite cookies, timeouts and invalidation at logout.
- **Test the real thing:** repeat the cookie tests against an authorised application with a real login (Section 6).

Limitations. Session validation, cookie attributes (Secure, HttpOnly, SameSite) and session expiry could not be tested because httpbin has no authentication.

1.0 Task Overview

Task ID	ASE-HTTP-SESSION
Task Title	HTTP & Session Security Analysis (TLS, Cookie Manipulation, Session Threat Modelling)
Skill Domain	Web Application Security / Session Management
Difficulty	Beginner
Duration	Per task brief (Phase F: 120 minutes)
CPE Credits	Per task brief
Tools Used	Kali Linux, curl 8.21.0, Burp Suite Community Edition v2026.3.2, Postman, TryHackMe, AI assistants (declared in Section 2.3)
Target Platform	httpbin.org (public HTTP request and response testing service); TryHackMe hosted labs
Guiding Framework(s)	OWASP Top 10:2025, CWE, 5×5 likelihood × impact risk matrix; OWASP Session Management Cheat Sheet (recommendations)
Regulatory Alignment	OWASP Top 10:2025 category mappings and CWE identifiers. No other standards were specified for this task.

1.1 Introduction

This task examines how HTTP, TLS and cookies work together to keep a user's session secure, and how weaknesses in them lead to session hijacking. It matters because a session cookie acts as the user's credential after login, so an Application Security Engineer must be able to test it and explain the risk. This report consolidates the Phase A–E tests into findings, a session-management threat model and recommendations.

1.2 Objectives & Learning Outcomes

The objectives of the task and how each was met:

- **Explain how TLS protects HTTP traffic** — Phase A: a written explanation plus curl captures of HTTPS (TLS 1.3, verified certificate) and plain HTTP.
- **Examine cookie modification and the server's response** — Phases C and D: modified cookies sent with Burp Suite Repeater and Postman; the server reflected the values.
- **Analyse session-security implications and mitigations** — Phase D: a security analysis with five mitigations.
- **Deepen HTTP knowledge in hosted labs** — Phase E: TryHackMe "HTTP in Detail" and "HTTP Request Smuggling".
- **Model session-management risk** — Phase F: threat model and a likelihood × impact matrix for session hijacking (Section 5).
- **Compile a professional report** — this document, with an executive summary, findings, risk matrix and recommendations.

1.3 Scope & Legal Authorisation

All activities were performed from a lab workstation against a public testing service and hosted training labs. Only benign requests (GET /get, /cookies and /cookies/set) were sent to the testing service; no attacks were run against any system.

Environments used:

- Kali Linux virtual machine (local) — analyst workstation running curl, Burp Suite and Postman
- httpbin.org — public HTTP request and response testing service
- TryHackMe — HTTP in Detail and HTTP Request Smuggling rooms

Confirmation: httpbin.org is the only third-party system that was contacted, and only with the benign requests listed above; no production networks were accessed. The Phase C explanation was AI-generated, and Claude was used for Phase F; both are declared in Section 2.3.

2.0 Environment & Tool Setup

2.1 Lab Architecture

The lab was one Kali Linux workstation running three HTTP clients (curl, Burp Suite and Postman), all sending requests to httpbin.org.

Component / VM	OS / Version	Role in Lab	Notes
Kali Linux VM	Kali Linux	Analyst workstation running all test tools	Prompt kali@kali; client address 10.0.2.15 (shown by curl)
curl	8.21.0	Compare HTTPS and HTTP traffic	`curl -v` against /get (Phase A)
Burp Suite Community Edition	v2026.3.2	Modify and resend a request (Repeater)	Target https://httpbin.org over HTTP/2 (Phase C)
Postman	Lightweight API Client	Cookie manipulation	Not signed in to a Postman account (Phase D)
httpbin.org	gunicorn/19.9.0 behind AWS ELB	Test target	Endpoints /get, /cookies, /cookies/set
TryHackMe	Hosted platform	Hosted labs (Phase E)	Profile abdoulkarimsavad

2.2 Tool Configuration

The two tools that needed configuration are described below. curl was run with its verbose flag only (see Phase A).

Tool Name	Burp Suite Community Edition v2026.3.2
Installation Method	Installed on the Kali Linux workstation
Configuration Steps	A GET /cookies request to https://httpbin.org was opened in Repeater (tab 3) and the Cookie header was changed from `session=12345` to `session=99999` before resending.
Verification	Response 200 OK over HTTP/2 with the cookie value reflected in the JSON body (Figure 3).

Tool Name	Postman (Lightweight API Client)
Installation Method	Installed on the Kali Linux workstation
Configuration Steps	Requests sent: GET `https://httpbin.org/cookies/set?session=attacker123`, then GET `https://httpbin.org/cookies`. No account or collection was used.
Verification	Both responses were 200 OK with `session: attacker123` in the cookies JSON (Figures 4 and 5).

2.3 AI Tool Usage Declaration

AI assistants were used in two places, as declared below.

AI Tool	Used For	What Was NOT Done with AI
AI assistant (tool not named in the Phase C file)	Generating the explanation of the Phase C result, labelled “AI Generated Explanation” in PhaseC.docx.	AI did not run Burp Suite, send the requests or take the screenshots.
Claude (Anthropic)	Phase F: structuring the report (Executive Summary, Findings, Recommendations), consolidating the Phase A–E evidence into this template, drafting and refining wording, and proposing likelihood and impact scores for the session threat model.	AI did not run the lab tests, produce the screenshots or the Phase A–E evidence files. The findings come from the curl, Burp Suite and Postman results.

The test results, screenshots and logs are the intern’s own work; AI assistance was used for explanation, structure and wording as declared above, and the intern has reviewed the report.

3.0 Methodology — Phase Execution

This section records what was done in each phase and what it showed.

Phase A — Theory & Conceptual Foundation

Phase Objective	Explain how TLS secures HTTPS and compare HTTP and HTTPS traffic with curl.
Estimated Time	Per task brief
Evidence File(s)	PhaseA.docx (Figures 1 and 2)

Actions Taken

I wrote a short explanation of TLS (Transport Layer Security) and its three roles: confidentiality (encrypting data between browser and server), authentication (certificates that let the browser verify the server) and integrity (detecting modification in transit). In summary, HTTPS = HTTP + TLS.

I then ran `curl -v https://httpbin.org/get` and `curl -v http://httpbin.org/get` from the Kali workstation to compare the two (Figures 1 and 2).

Key Observations

Over HTTPS, curl negotiated TLS 1.3 with the cipher TLS_AES_128_GCM_SHA256 and the X25519MLKEM768 key exchange. The server presented a certificate for httpbin.org (RSA 2048, issued by Amazon RSA 2048 M01, valid 19 Jun 2026 to 2 Jan 2027) that OpenSSL verified (verify result 0). This shows the three TLS roles in practice. The HTTPS request itself returned 503 Service Temporarily Unavailable from the load balancer (server: awselb/2.0), so no application response was captured over HTTPS.

The plain HTTP request to the same host returned 200 OK with no redirect to HTTPS. The headers and the JSON body were visible in clear text, including `Server: gunicorn/19.9.0`, `Access-Control-Allow-Origin: *` and `Access-Control-Allow-Credentials: true`. Anyone on the network path could read and alter this traffic, and a session cookie sent this way would be exposed.

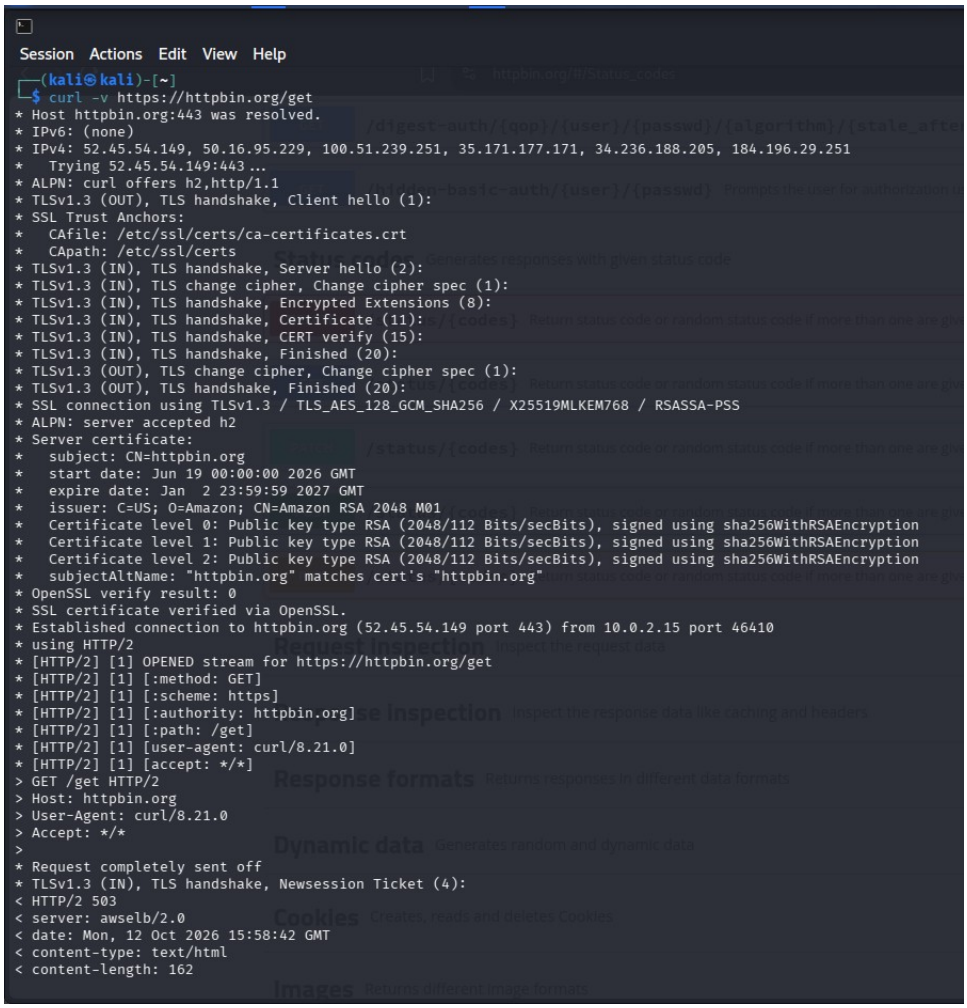


Figure 1 — `curl -v https://httpbin.org/get`: TLS 1.3 handshake, verified certificate and a 503 response from the load balancer.

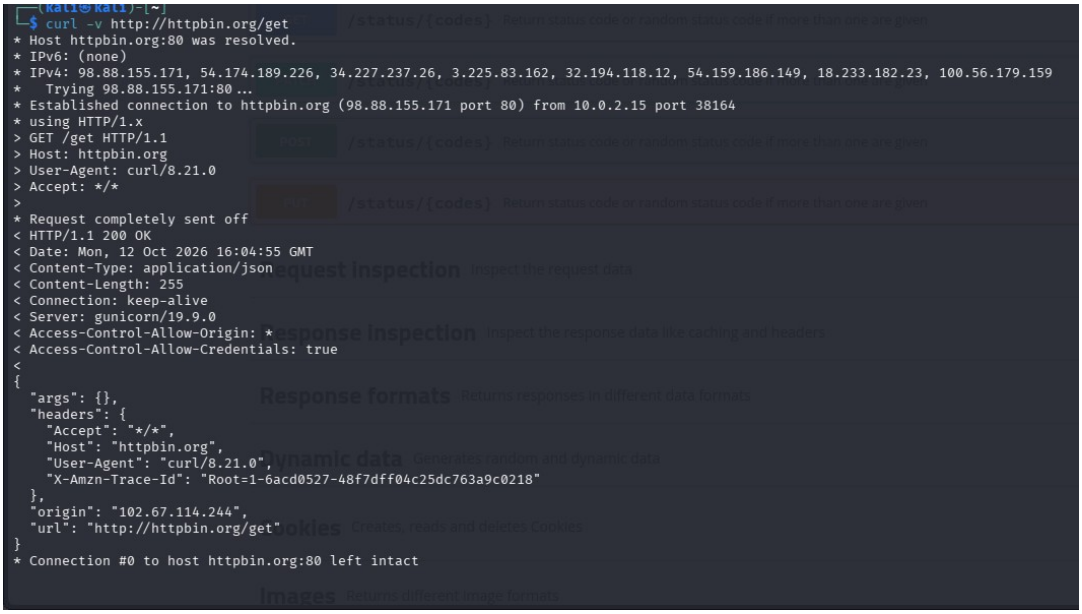


Figure 2 — `curl -v http://httpbin.org/get`: 200 OK in clear text, with Server, CORS headers and the JSON body visible.

Evidence Produced

Evidence files and artefacts for this phase:

- PhaseA.docx — TLS explanation and the two curl captures
- Figure 1 — `curl -v` over HTTPS; Figure 2 — `curl -v` over HTTP

Phase B — Lab Setup & Environment Preparation

Phase Objective	Set up Burp Suite Repeater on the Kali workstation and capture a request to prepare for the cookie tests.
Estimated Time	Per task brief
Evidence File(s)	Phase B screenshot (Figure B1)

Actions Taken

Burp Suite Community Edition v2026.3.2 was running on the Kali Linux workstation with a Temporary Project. In the Repeater tab (tab 3) I set the target to https://httpbin.org (HTTP/2) and prepared a `GET /cookies` request with a `Cookie: session=99999` header, then sent it (Figure B1).

Key Observations

The request returned `HTTP/2 200 OK` (249 bytes, 219 ms) with six response headers and the JSON body `{"cookies": {"session": "99999"}}`. Burp's Inspector shows one request cookie and 18 request headers. This shows that the tool and request were working; the same capture is analysed in Phase C. The phase objective is not stated in the evidence, so I described it from what the screenshot shows.

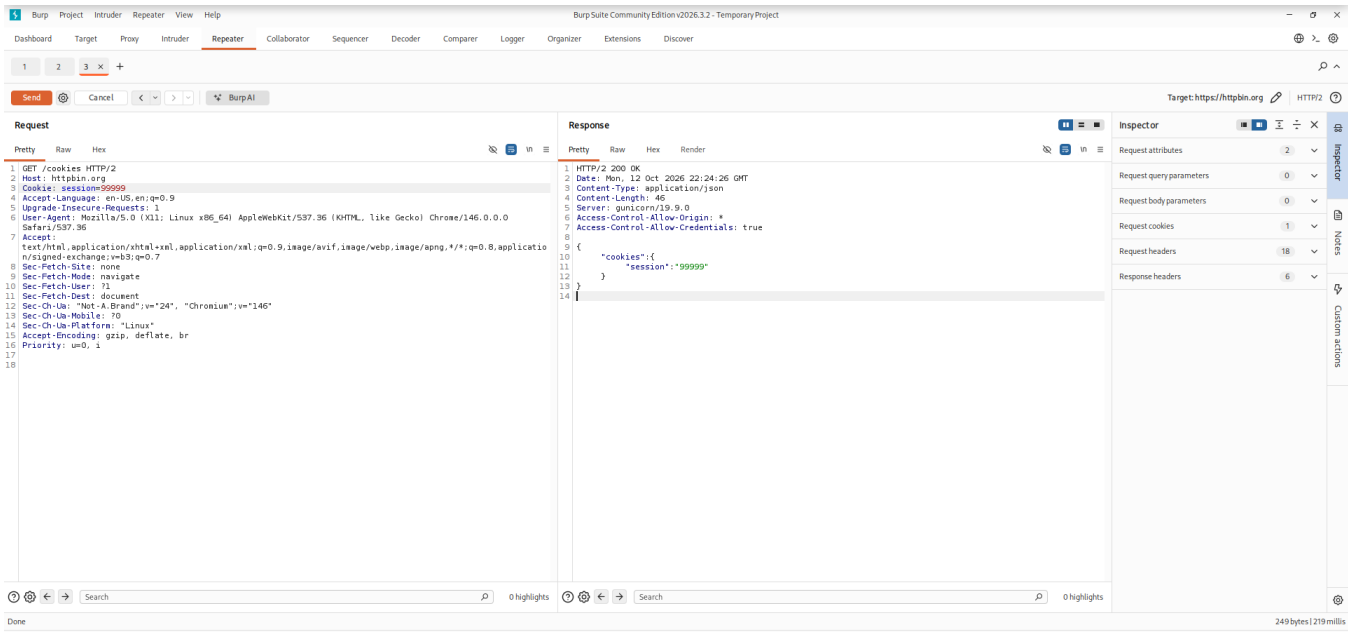


Figure B1 — Burp Suite Community Edition v2026.3.2, Repeater tab 3: `GET /cookies` with `Cookie: session=99999` (left), the 200 OK response (centre) and the Inspector (right).

Evidence Produced

Evidence files and artefacts for this phase:

- Phase B screenshot — Burp Suite Repeater (full window)
- Figure B1 — Burp Suite Repeater, tab 3, target https://httpbin.org

Phase C — Core Technical Execution

Phase Objective	Examine cookie modification and observe the server's response.
Estimated Time	Per task brief
Evidence File(s)	PhaseC.docx (Figure 3)

Actions Taken

In Burp Suite Community Edition (Repeater) I captured a GET /cookies request to https://httpbin.org, changed the Cookie header from `session=12345` to `session=99999` and resent it (Figure 3). An AI-generated explanation of the result is included in the Phase C file and labelled as such.

Key Observations

The server returned 200 OK over HTTP/2 with the JSON body `{"cookies": {"session": "99999"}}`, so the modified value was transmitted and reflected. The /cookies endpoint does not authenticate anyone, so this does not show that a modified value would be accepted as a valid session; it shows only that the client controls what it sends.

The Secure, HttpOnly and SameSite attributes could not be assessed: the test sent a Cookie header, and no Set-Cookie header was captured.

Evidence Produced

Evidence files and artefacts for this phase:

- PhaseC.docx — Burp Repeater screenshot and AI-generated explanation
- Figure 3 — Burp Suite Repeater request and response

Detailed Technical Walkthrough

The test details, outputs, decisions and limitations behind the Phase C result:

- **Request:** Burp Suite Community Edition v2026.3.2, Repeater tab 3, target https://httpbin.org (HTTP/2). `GET /cookies HTTP/2`, `Host: httpbin.org`, `Cookie: session=99999` (original value `session=12345`).
- **Response:** `HTTP/2 200 OK`; Date Mon, 12 Oct 2026 22:24:26 GMT; Content-Type application/json; Content-Length 46; `Server: gunicorn/19.9.0`; `Access-Control-Allow-Origin: *`; `Access-Control-Allow-Credentials: true`; body `{"cookies": {"session": "99999"}}` (Figure 3).
- **Decision:** I changed a cookie value rather than attempting an attack, because the target does not validate sessions. The conclusion therefore stays at "a client-controlled value was reflected" and is rated Informational (Finding 2).
- **Limitation:** the endpoint has no authentication, so session validation could not be assessed. Further testing against an authorised application with a real login is required.

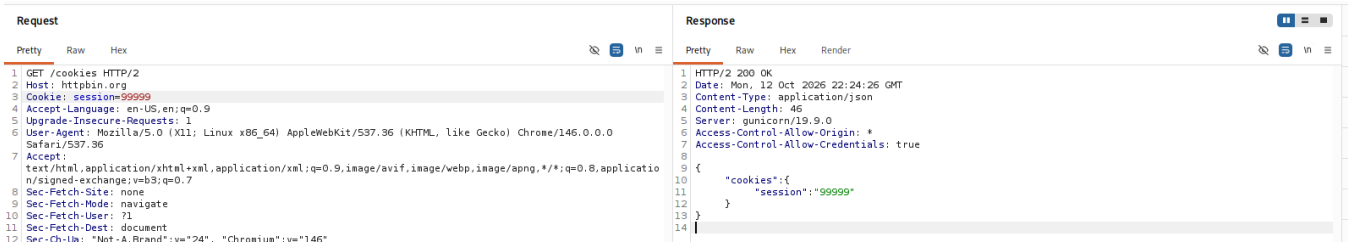


Figure 3 — Burp Suite Repeater: request with `Cookie: session=99999` (left) and the 200 OK response reflecting it (right).

Phase D — Independent Extension & Threat Modelling

Phase Objective	Manipulate a cookie with a second tool and analyse the session-security implications and mitigations.
Estimated Time	Per task brief
Evidence File(s)	PhaseD.docx (Figures 4 and 5)

Actions Taken

In Postman I sent GET `https://httpbin.org/cookies/set?session=attacker123`, then GET `https://httpbin.org/cookies` (Figures 4 and 5). Both returned 200 OK with `{"cookies": {"session": "attacker123"}}`. I then wrote a security analysis and a list of five mitigations.

Key Observations

The server set and returned an arbitrary value chosen by the client, the same behaviour seen in Phase C with a different tool. The value was also placed in the URL query string, where it can end up in logs, browser history and Referer headers (threat T7).

As in Phase C, this shows that a client can supply any cookie value; it does not establish a session-management vulnerability, because httpbin has no login. In a real application the same weakness could allow session fixation, and predictable or unvalidated IDs could expose accounts.

Phase D mitigations: (1) regenerate session IDs after login and privilege changes; (2) use cryptographically secure, unpredictable IDs; (3) validate session IDs and authentication state on the server; (4) set Secure, HttpOnly and SameSite; (5) enforce expiry and invalidate sessions at logout.

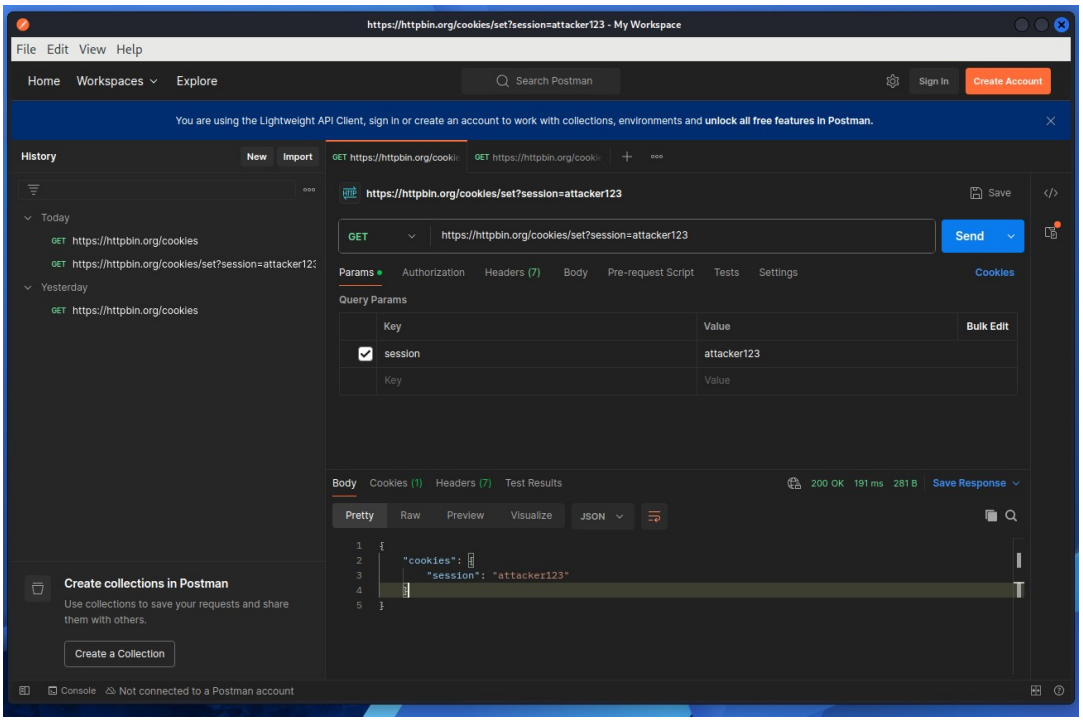


Figure 4 — Postman: GET /cookies/set?session=attacker123 returns 200 OK with the cookie reflected.

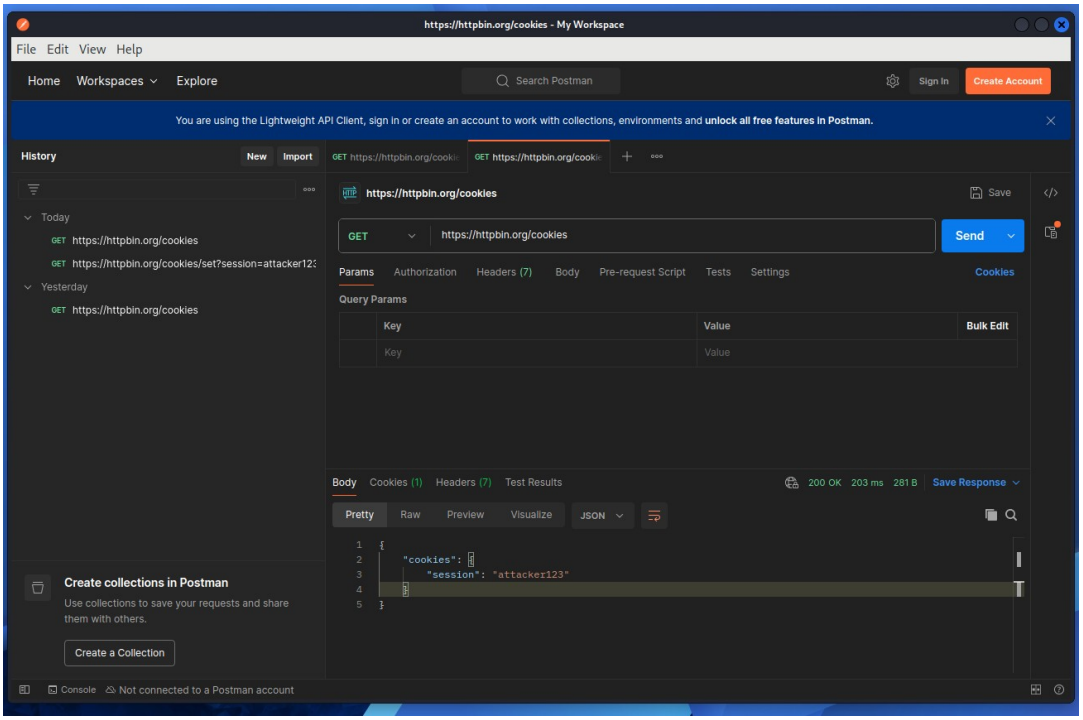


Figure 5 — Postman: GET /cookies returns 200 OK with session = attacker123.

Evidence Produced

Evidence files and artefacts for this phase:

- PhaseD.docx — two Postman screenshots, analysis and mitigations
- Figures 4 and 5 — Postman requests and responses

Phase E — Hosted Lab Integration

Phase Objective	Reinforce HTTP knowledge through hosted TryHackMe labs.
Estimated Time	Per task brief
Evidence File(s)	PhaseE-Lab_completion_screenshots_and_summary_paragraph.docx (Figure 6)

Actions Taken

I completed two TryHackMe labs (Figure 6). In HTTP in Detail I covered HTTP and HTTPS, request and response structure, methods and their purposes, status codes, headers and cookies as the way state and user sessions are kept across requests. In HTTP Request Smuggling I covered what it is, how attackers exploit differences in how servers or intermediaries interpret requests, the three types (CL.TE, TE.CL, TE.TE) and a practical example.

Key Observations

Lab 1 explains the mechanism the cookie tests relied on: HTTP is stateless, and cookies are how it carries a session. Lab 2 adds a threat the cookie tests did not cover: if a front-end and a back-end disagree on where a request ends, an attacker can smuggle a request prefix that is joined to another user's request, which can expose that user's headers and session cookie (threat T10). This was studied in the lab, not tested against the httpbin target.

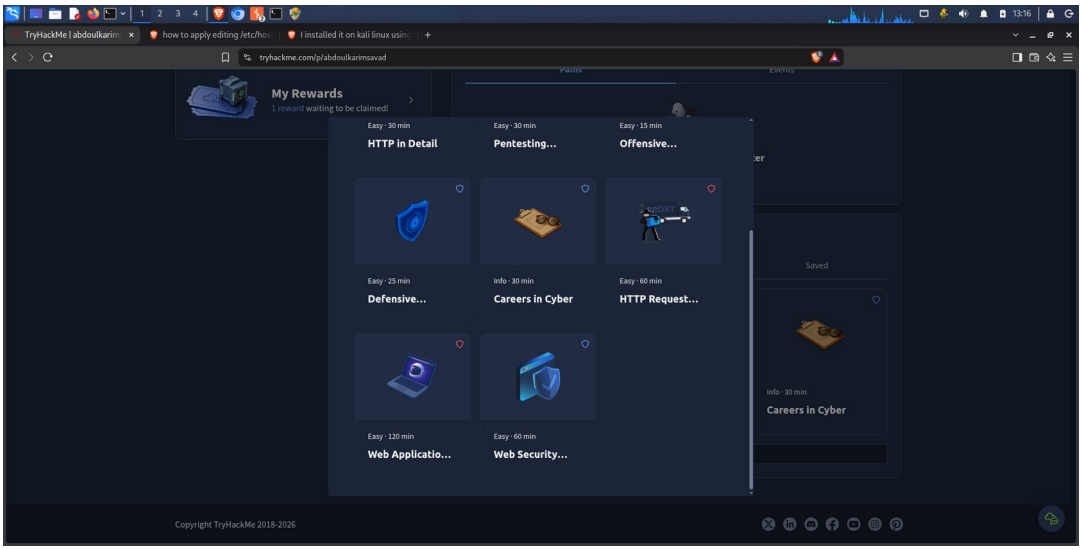


Figure 6 — TryHackMe (abdoulkarimsavad): the HTTP in Detail and HTTP Request Smuggling rooms.

Evidence Produced

Evidence files and artefacts for this phase:

- PhaseE-Lab_completion_screenshots_and_summary_paragraph.docx — lab screenshot and summary
- Figure 6 — TryHackMe room list showing HTTP in Detail and HTTP Request Smuggling

Phase F — Documentation & Reporting

Phase Objective	Compile all findings into a professional report.
Estimated Time	120 minutes
Evidence File(s)	ASE_HTTP_Session_Report.pdf

Actions Taken

I consolidated the screenshots, logs and AI summaries from Phases A, B, C, D and E into the EncryptEdge Labs report template. Claude helped structure the report and refine the wording, as declared in Section 2.3. I added a session-management threat model (Section 5.1), a likelihood × impact risk matrix for session hijacking (Section 5.2, Figure 7), a one-page executive summary and mitigation strategies, and exported the report as ASE_HTTP_Session_Report.pdf.

Key Observations

The tests only show that a client controls its cookie and that the service answers over plain HTTP. They do not show a working hijack, so the threat model scores likelihood and impact for a typical cookie-based application, using the lab results as evidence for the enabling conditions. Those scores are the author's assessments, listed with their basis in Section 5.2.

Phase B is documented by one Burp Suite Repeater screenshot (Figure B1).

Evidence Produced

Evidence files and artefacts for this phase:

- ASE_HTTP_Session_Report.pdf — this report

4.0 Findings

Four findings are recorded, all from the Phase A, C and D tests against httpbin.org. Because httpbin is a test service that issues no real sessions, each finding states what it would mean in a real application that issues session cookies. Threats that were not observed in the lab, such as XSS cookie theft and session fixation, are in the threat model (Section 5) rather than listed here as findings.

4.1 Findings Summary

#	Finding Title	Type / Category	Framework Ref.*	Phase Identified	Severity / Impact	Evidence Ref.
1	Plaintext HTTP accepted, no redirect to HTTPS	Transport security (confirmed, curl)	A04:2025 / CWE-319	Phase A	Medium	Fig. 2
2	Client-supplied cookie values accepted and reflected	Session management (observation)	A07:2025 / CWE-565	Phases C, D	Informational	Fig. 3–5
3	Server software version disclosed	Security Misconfiguration (confirmed)	A02:2025 / CWE-200	Phases A, C	Low	Fig. 1–3
4	Wildcard CORS with credentials header	Security Misconfiguration (confirmed)	A02:2025 / CWE-942	Phases A, C	Low	Fig. 2–3

* Framework Ref: OWASP Top 10:2025 category numbers and CWE identifiers.

4.2 Detailed Finding Analysis

Detailed analysis of the four findings.

Finding 1: Plaintext HTTP accepted, no redirect to HTTPS

Finding Title	Plaintext HTTP accepted, no redirect to HTTPS
Type / Category	Transport security (confirmed by curl)
Description	On `curl -v http://httpbin.org/get` the service answered `HTTP/1.1 200 OK` on port 80 and returned its headers and JSON body in clear text, with no redirect to HTTPS (Figure 2). The HTTPS request to the same host returned a 503 from the load balancer at the time (Figure 1), so HSTS could not be assessed over HTTPS.
How Identified	curl -v over HTTP and over HTTPS from the Kali workstation (Phase A).
Framework Reference	OWASP A04:2025 Cryptographic Failures / CWE-319 (cleartext transmission of sensitive information)
Business / Operational Impact	If this were an application issuing session cookies, an attacker on the network path (public Wi-Fi, a compromised router) could read the cookie from a cleartext request and replay it to hijack the session (confidentiality, integrity). httpbin sets no real sessions, so the practical impact here is nil; the finding shows what to check on a real target.
Severity / Impact	Medium (High if the service issues session cookies)
Evidence Reference	Figure 2; PhaseA.docx

Finding 2: Client-supplied cookie values accepted and reflected

Finding Title	Client-supplied cookie values accepted and reflected
Type / Category	Session management (observation)
Description	A cookie value chosen by the client was accepted and returned by the server: `session=99999` sent through Burp Repeater (Figure 3) and `session=attacker123` sent through Postman (Figures 4 and 5). Both requests returned 200 OK with the same value in the JSON body.
How Identified	Burp Suite Repeater (Phase C) and Postman (Phase D).
Framework Reference	OWASP A07:2025 Authentication Failures (session management) / CWE-565 (reliance on cookies without validation)
Business / Operational Impact	Shows that the cookie is entirely client-controlled. If a real application trusted such a value without server-side validation or an integrity check, an attacker could impersonate users or fix a known session ID on a victim. httpbin has no authentication, so this is not itself a vulnerability and no bypass was demonstrated.
Severity / Impact	Informational
Evidence Reference	Figures 3–5; PhaseC.docx, PhaseD.docx

Finding 3: Server software version disclosed

Finding Title	Server software version disclosed
Type / Category	Security Misconfiguration (confirmed)
Description	Responses disclosed `Server: gunicorn/19.9.0` on the 200 responses (Figures 2 and 3) and `server: awselb/2.0` on the HTTPS 503 (Figure 1).
How Identified	Response headers in curl output and in Burp Suite.
Framework Reference	OWASP A02:2025 Security Misconfiguration / CWE-200
Business / Operational Impact	Version banners help an attacker choose exploits for that software (confidentiality, low). Gunicorn 19.9.0 is an older release; known vulnerabilities for it were not looked up.
Severity / Impact	Low
Evidence Reference	Figures 1–3

Finding 4: Wildcard CORS with credentials header

Finding Title	Wildcard CORS with credentials header
Type / Category	Security Misconfiguration (confirmed)
Description	Responses carried `Access-Control-Allow-Origin: *` together with `Access-Control-Allow-Credentials: true` (Figures 2 and 3).
How Identified	Response headers in curl output and in Burp Suite.
Framework Reference	OWASP A02:2025 Security Misconfiguration / CWE-942 (permissive cross-domain policy)
Business / Operational Impact	Browsers refuse to share credentialed responses under a wildcard origin, which limits the risk. The pattern still signals a CORS policy that was not designed; a variant that reflects the caller's origin would let any site read authenticated responses on a real application (confidentiality).
Severity / Impact	Low
Evidence Reference	Figures 2–3

5.0 Risk Assessment & Threat Model

This section models the threats to session management, scores the risk of session hijacking on a likelihood × impact matrix, and sets out the controls and the exposure if the risks stay open. The lab provides evidence for some enabling conditions (plaintext HTTP, client-controlled cookies); the likelihood and impact scores are the author's assessments for a typical cookie-based web application.

5.1 Threat Model: Session Management

Assets: S1 session identifier (cookie); S2 user account and authenticated state; S3 server-side session store; S4 transport channel between client and server. **Method:** list the ways each asset can be attacked, then link each threat to what the lab showed.

Threat	Attack path	Asset	Basis in this lab
T1 Sniffing or tampering over cleartext HTTP	An attacker on the network path reads or alters HTTP traffic and copies the session cookie	S1, S4	Finding 1: httpbin answers on plain HTTP with 200 OK (Figure 2)
T2 Cookie theft via XSS	Injected script reads `document.cookie` when the cookie lacks HttpOnly	S1, S2	Cookie attributes were not captured, so unverified
T3 Session fixation	The attacker sets a known ID before login and the ID is not regenerated afterwards	S1, S2	Phase D: the server accepted a client-chosen cookie (Figures 4, 5)
T4 Session ID prediction or brute force	Valid IDs are guessed when they are short or predictable	S1	Test values such as 12345 and 99999 show how guessable formats look (lab values only)
T5 Cookie tampering	The client edits the cookie and the server trusts it without validation or integrity checks	S1, S2, S3	Finding 2: 99999 and attacker123 were reflected (Figures 3–5)
T6 CSRF (session riding)	The browser attaches the session cookie to a cross-site request	S2	SameSite attribute not captured, so unverified
T7 Session ID leakage	The ID appears in URLs, logs, history or Referer headers	S1	Phase D put `session=attacker123` in the request URL (Figure 4)
T8 Weak expiry, no logout invalidation	A stolen ID remains valid for a long time or after logout	S1, S3	Not testable: httpbin has no login
T9 Permissive CORS with credentials	Other origins read authenticated responses	S2, S4	Finding 4: wildcard origin with allow-credentials (Figures 2, 3)
T10 Request smuggling	Front-end and back-end disagree on request boundaries, so an attacker's prefix captures another user's request and cookie	S1, S4	Phase E lab (CL.TE, TE.CL, TE.TE); studied, not tested on the target

5.2 Risk Matrix: Session Hijacking

Likelihood (L) and impact (I) are scored 1–5 and the risk score is L × I (Figure 7). Higher likelihood reflects what the lab confirmed (T1: plaintext HTTP is served; T5: client-chosen cookies are accepted) and the common occurrence of the attack in practice. Impact is 4–5 where a stolen or forged session gives account takeover. Assets are S1–S4 from Section 5.1.

Threat / Risk	Business Asset	Framework Ref.	Likelihood 1–5	Impact 1–5	Score	Rating
T1 Sniffing / tampering over HTTP	S1, S4	A04:2025 / CWE-319	4	5	20	Critical
T2 Cookie theft via XSS	S1, S2	A05:2025 / CWE-79, CWE-1004	3	5	15	Critical
T5 Cookie tampering	S1, S2, S3	A07:2025 / CWE-565	3	4	12	High
T8 Weak expiry / no logout invalidation	S1, S3	A07:2025 / CWE-613	3	4	12	High
T4 Session ID prediction	S1	A07:2025 / CWE-330	2	5	10	High
T10 Request smuggling	S1, S4	CWE-444	2	5	10	High
T6 CSRF	S2	A01:2025 / CWE-352	3	3	9	Medium
T7 Session ID leakage	S1	A04:2025 / CWE-598	3	3	9	Medium
T3 Session fixation	S1, S2	A07:2025 / CWE-384	2	4	8	Medium
T9 Permissive CORS with credentials	S2, S4	A02:2025 / CWE-942	2	3	6	Medium

Score guide: 1–4 = Low | 5–9 = Medium | 10–14 = High | 15–25 = Critical

Result: 2 Critical, 4 High and 4 Medium. The two Critical risks are the paths that hand an attacker a working session cookie with little effort: reading it off the network, or stealing it with injected script.

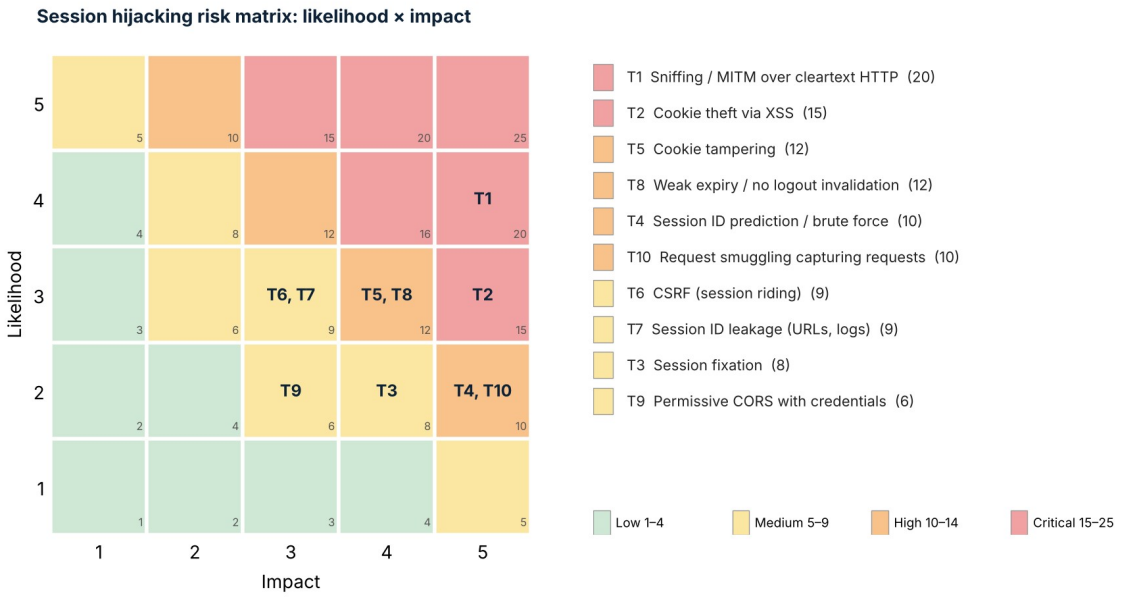


Figure 7 — Risk matrix for session hijacking: likelihood × impact for threats T1–T10 (score in each cell's corner).

5.3 Defensive Opportunities & Coverage Gaps

Where a control could prevent, detect or reduce each threat:

Finding / Gap	Observable / Trigger	Recommended Control	Tool / Implementation
T1 Cleartext HTTP	Requests on port 80; cookies sent without Secure; one session used from two networks	HTTPS only: redirect or close port 80, HSTS, Secure flag	Reverse proxy or CDN configuration; HSTS preload; ZAP baseline check
T2, T6 XSS theft and CSRF	Scripts in inputs; CSP violation reports; cross-site POSTs	HttpOnly and SameSite cookies; CSP; output encoding	Framework cookie settings; CSP header; SAST
T3, T4 Fixation and prediction	Same ID before and after login; many unknown IDs from one IP	Regenerate the ID at login; 128-bit random IDs; rate limiting	Session middleware; WAF rate rules; SIEM alert
T5 Cookie tampering	Cookies that fail the lookup or signature check	Server-side lookup or HMAC-signed cookies; reject unknown IDs	Session store (for example Redis); signed-cookie library
T7 ID leakage	Session IDs in URLs or access logs	Carry IDs in cookies only; scrub logs; Referrer-Policy	Log scrubbing; log review; Referrer-Policy header
T8 Weak expiry	Sessions older than policy; sessions still valid after logout	Idle and absolute timeouts; revoke at logout and password change	Session store TTL; automated logout test
T9 Permissive CORS	`Access-Control-Allow-Origin: *` with credentials; unexpected Origin headers	Explicit origin allow-list; no wildcard on session endpoints	CORS middleware; ZAP header check
T10 Request smuggling	Requests with both Content-Length and Transfer-Encoding; desynchronised responses	Consistent parsing, HTTP/2 end to end, reject ambiguous requests, patch proxies	Reverse-proxy settings; Burp Suite scanning and HTTP Request Smuggler extension

5.4 Risk Exposure Over Time

What happens if the three most serious risks stay open:

Risk 1 — Sniffing over cleartext HTTP (T1): while plain HTTP is served, anyone on the same network path can read cookies and headers, and a cookie captured once can be replayed until it expires, giving full access to the account. With no HTTPS redirect or HSTS, users who type the address or follow an old http:// link are exposed on every visit, so the risk grows with the number of users on untrusted networks.

Risk 2 — Cookie theft and tampering (T2, T5): if session cookies lack HttpOnly, any XSS flaw lets an attacker read them and hijack sessions at scale. If the server also trusts a client-controlled cookie without validation, tampering extends to impersonation. The Phase C and D tests could not confirm the cookie attributes, so this exposure stays unknown, and unknown risk tends to persist.

Risk 3 — Weak expiry and logout invalidation (T8): without idle and absolute timeouts and server-side invalidation, a session ID that leaks by any route (T1, T2, T7, T10) stays usable indefinitely. A one-off leak becomes long-term access, and incident response is harder because the ID cannot be revoked.

6.0 Recommendations

Each recommendation traces to a finding in Section 4 or a threat in Section 5. Priority follows the risk rating: Critical = Immediate, High and Medium = Short-term, Low = Long-term.

6.1 Technical Remediation

#	Finding	Specific Remediation Action	Priority	Standard / Reference
1	Plaintext HTTP accepted (T1)	Redirect all port 80 traffic to HTTPS with a 301, or close port 80. Send `Strict-Transport-Security: max-age=31536000; includeSubDomains` and set the Secure attribute on every session cookie.	Immediate	OWASP Transport Layer Security Cheat Sheet / CWE-319
2	Client-supplied cookie values (T3, T4, T5)	Issue opaque random session IDs (at least 128 bits from a CSPRNG) and look them up on the server, or sign cookies with an HMAC. Reject unknown IDs and regenerate the ID after login and privilege changes.	Short-term	OWASP Session Management Cheat Sheet / CWE-565
3	Cookie attributes unverified (T2, T6)	Capture the Set-Cookie header of a real login and confirm Secure, HttpOnly and SameSite=Lax or Strict, a narrow Path and Domain, and a bounded lifetime. Add a CSP to limit XSS.	Immediate	OWASP Session Management Cheat Sheet / CWE-1004
4	Session lifecycle (T7, T8)	Enforce idle and absolute timeouts, invalidate the server-side session at logout and password change, and never carry session IDs in URLs.	Short-term	CWE-613, CWE-598
5	Wildcard CORS with credentials	Never combine `Access-Control-Allow-Origin: *` with `Access-Control-Allow-Credentials: true`. Use an explicit origin allow-list and omit CORS headers on session-bearing endpoints.	Long-term	A02:2025 / CWE-942
6	Server version disclosed	Suppress or genericise the Server header at the reverse proxy, for example nginx `server_tokens off;` and `proxy_hide_header Server;`.	Long-term	A02:2025 / CWE-200
7	Request smuggling (T10)	Use HTTP/2 end to end where possible, reject requests that carry both Content-Length and Transfer-Encoding, and keep front-end and back-end parsers consistent and patched.	Short-term	CWE-444

6.2 Strategic & Operational Improvements

Beyond the individual fixes, these improvements would prevent similar weaknesses from recurring:

- **Test with a real login:** repeat the Phase C and D cookie tests against an authorised application with real authentication, because httpbin does not validate sessions and the results could not show whether session controls hold.

- **Automate session-security checks in CI/CD:** fail the build if HTTP is served without a redirect, HSTS is missing, or session cookies lack Secure, HttpOnly or SameSite (for example with a ZAP baseline scan or header tests).
- **Adopt a session-management standard:** build and review sessions against the OWASP Session Management Cheat Sheet and the session requirements in OWASP ASVS.
- **Detect session abuse:** log and alert on one session ID used from two networks or user agents, bursts of invalid session IDs, and logins that reuse a pre-login ID.
- **Review proxy chains for parsing consistency:** include request-smuggling checks (Phase E) whenever a new front-end proxy or CDN is added in front of an application.

7.0 Reflection

Reflections on the assessment, grounded in the tests above.

Q1 Which session threat presents the greatest risk, and why?

Sniffing or tampering over cleartext HTTP (T1, score 20). It needs no flaw in the application: if a cookie crosses the network unencrypted, anyone on the path can copy it, and the lab confirmed that httpbin answers on plain HTTP. It is also one of the cheapest to fix, with an HTTPS redirect, HSTS and the Secure flag. The matrix is my assessment for a typical application, since no hijack was carried out.

Q2 What did the cookie experiments prove, and what did they not prove?

They proved that the client decides what cookie value it sends, and that httpbin returns it unchanged, in both Burp Suite and Postman. They did not prove a session-management vulnerability, because httpbin has no login and does not validate sessions. They also could not show the Secure, HttpOnly and SameSite attributes, session expiry or regeneration after login.

Q3 How would you explain the key risks to a non-technical stakeholder?

A session cookie is like a wristband that proves you already logged in. If it travels over an unencrypted connection, or a malicious script can read it, someone else can wear your wristband and act as you. The fixes are well known: encrypt every connection, make the wristband unguessable and checked by our server, hide it from scripts, and make it expire. Today’s tests showed some enabling conditions, such as an unencrypted option being available, but not a working takeover.

Q4 What would you do differently or next?

I would repeat the tests against an authorised application with a real login, capture the Set-Cookie header to check the cookie attributes, and test session regeneration, expiry and logout. I would also capture the HTTPS response when the service is available, to check HSTS, and keep a log of each Repeater request.

8.0 Deliverables Submission Checklist

Deliverables for each phase, with the evidence filenames used in this report. The Phase F checklist items are all met: threat model and risk matrix included (Section 5), executive summary written (page 2), report exported as PDF.

✔	Phase	Deliverable Description	Your Filename	Status
☑️	Phase A	TLS explanation and curl captures of HTTPS and HTTP (Figures 1, 2)	PhaseA.docx	Complete
☑️	Phase B	Burp Suite Repeater setup and request (Figure B1)	Phase B screenshot	Complete
☑️	Phase C	Burp Suite cookie-modification test and explanation (Figure 3)	PhaseC.docx	Complete
☑️	Phase D	Postman cookie manipulation, analysis and mitigations (Figures 4, 5)	PhaseD.docx	Complete
☑️	Phase E	TryHackMe lab screenshot and summary (Figure 6)	PhaseE-Lab_completion_screenshots_and_summary_paragraph.docx	Complete
☑️	Phase F	This report, exported as PDF	ASE_HTTP_Session_Report.pdf	Complete

Submission naming:

- Report PDF: ASE_HTTP_Session_Report.pdf (name given in the Phase F brief)
- Evidence files: PhaseA.docx, PhaseC.docx, PhaseD.docx, Phase B screenshot (Burp Suite Repeater), PhaseE-Lab_completion_screenshots_and_summary_paragraph.docx

9.0 Additional Observations

Additional observations from the task:

- **Tool behaviour:** the HTTPS request to httpbin.org returned 503 from the AWS load balancer at 15:58 GMT, while the HTTP request about six minutes later returned 200. The two responses are therefore not a like-for-like comparison, and the HTTPS application response and its headers were not captured.
- **Privacy:** the Figure 2 screenshot shows the workstation's public IP address in httpbin's `origin` field. Redact it if the report is shared beyond the programme.
- **Scoring notes:** the likelihood and impact scores in Section 5.2 are the author's assessments for a typical cookie-based application.
- **Phase B scope:** the Phase B evidence is one screenshot, and it shows the same Burp Repeater request as Phase C. The phase objective was not stated, so Section 4 describes it from the screenshot.

Declaration & Sign-Off

I confirm that:

- All work in this report is my own except where AI usage is explicitly declared in Section 2.3.
- All technical activities were performed exclusively within the authorised environments listed in Section 1.3.
- No production networks or third-party infrastructure were accessed during this task, other than benign requests to the public testing service httpbin.org described in Section 1.3.
- I have reviewed this report for accuracy, completeness, and professional presentation before submission.

Intern Full Name	Signature	Date of Submission
Abdoul Karim Savadogo		15 October 2026

EncryptEdge Labs Limited — Junior Application Security Engineer Internship Programme
Company No. 15830711 | United Kingdom | encryptedgelabs.com

Copyright © 2026 EncryptEdge Labs. All rights reserved. | Report Template v3.0